

Exercise 2: Product Search (Linear Search & Binary Search) – C#

Folder Structure

DSA_Module

|

└─ FinancialForecast

|

└─ ProductSearch

|

└─ Product.cs

└─ Program.cs

└─ ProductSearch.csproj

File Name: Product.cs

```
using System;
```

```
public class Product
```

```
{
```

```
    public int ProductId;
```

```
    public string ProductName;
```

```
    public string Category;
```

```
    public Product(int productId, string productName, string category)
```

```
    {
```

```
        ProductId = productId;
```

```
        ProductName = productName;
```

```
        Category = category;
```

```
    }
```

```
    public void DisplayProduct()
```

```
    {
```

```
        Console.WriteLine($"Product ID: {ProductId}");
```

```
        Console.WriteLine($"Product Name: {ProductName}");  
        Console.WriteLine($"Category: {Category}");  
    }  
}
```

File Name: Program.cs

```
using System;  
  
class Program  
{  
    static Product LinearSearch(Product[] products, int searchId)  
    {  
        foreach (Product product in products)  
        {  
            if (product.ProductId == searchId)  
                return product;  
        }  
  
        return null;  
    }  
  
    static Product BinarySearch(Product[] products, int searchId)  
    {  
        int left = 0;  
        int right = products.Length - 1;  
  
        while (left <= right)  
        {  
            int mid = (left + right) / 2;  
  
            if (products[mid].ProductId == searchId)
```

```

        return products[mid];

    if (products[mid].ProductId < searchId)
        left = mid + 1;
    else
        right = mid - 1;
    }

    return null;
}

static void Main()
{
    Product[] products =
    {
        new Product(101, "Laptop", "Electronics"),
        new Product(102, "Shoes", "Fashion"),
        new Product(103, "Mobile", "Electronics"),
        new Product(104, "Watch", "Accessories"),
        new Product(105, "Book", "Education")
    };

    int searchId = 104;

    Console.WriteLine("Linear Search Result:");
    Product linearResult = LinearSearch(products, searchId);

    if (linearResult != null)
        linearResult.DisplayProduct();
    else
        Console.WriteLine("Product not found");
}

```

```
Console.WriteLine();

Console.WriteLine("Binary Search Result:");
Product binaryResult = BinarySearch(products, searchId);

if (binaryResult != null)
    binaryResult.DisplayProduct();
else
    Console.WriteLine("Product not found");

Console.WriteLine();
Console.WriteLine("Linear Search: O(n)");
Console.WriteLine("Binary Search: O(log n)");
}
}
```

Run Command

dotnet run

Expected Output

Linear Search Result:

Product ID: 104

Product Name: Watch

Category: Accessories

Binary Search Result:

Product ID: 104

Product Name: Watch

Category: Accessories

Linear Search: $O(n)$

Binary Search: $O(\log n)$

Explanation

Linear Search:

- Searches elements one by one.
- Time Complexity: $O(n)$

Binary Search:

- Works on sorted data.
- Divides search space into halves.
- Time Complexity: $O(\log n)$

Binary Search is faster for large sorted datasets.

The screenshot shows the Visual Studio Code interface. The main editor displays a C# file named `Program.cs` with the following code:

```
using System;

public class FinancialForecast
{
    public static double CalculateFutureValue(double currentValue, double growthRate, int years)
    {
        if (years == 0)
        {
            return currentValue;
        }

        return CalculateFutureValue(currentValue * (1 + growthRate), growthRate, years - 1);
    }

    static void Main(string[] args)
    {
        double currentValue = 10000;
        double growthRate = 0.10;
        int years = 5;

        double futureValue = CalculateFutureValue(currentValue, growthRate, years);

        Console.WriteLine($"Current Value: {currentValue}");
        Console.WriteLine($"Growth Rate: {growthRate * 100}%");
        Console.WriteLine($"Years: {years}");
        Console.WriteLine($"Future Value: {futureValue}");
    }
}
```

The Solution Explorer on the right shows the project structure:

- DSA_Module (C:\Users\vanka\OneDrive\Desktop\DSA)
 - FinancialForecast
 - bin
 - obj
 - ProductSearch
 - bin
 - obj
 - Product.cs
 - ProductSearch.csproj
 - Program.cs
 - FinancialForecast.csproj

The Developer PowerShell terminal at the bottom shows the output of the program:

```
Binary Search Result:
Product ID: 104
Product Name: Watch
Category: Accessories
PS C:\Users\vanka\OneDrive\Desktop\DSA_Module\FinancialForecast\ProductSearch>
```